

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ  
ФЕДЕРАЦИИ федерального государственного бюджетного  
образовательного учреждения высшего образования  
«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ  
Г.В.ПЛЕХАНОВА»  
Техникум Пермского института (филиала)

**Практическая работа №3**  
по дисциплине «Поддержка и тестирование программных модулей»  
Тема: «Разработка приложения управления складами»  
Студента группы ИПо-32  
Светлышева Елизавета Денисовна

2026 г.

## **Оглавление**

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>Введение .....</b>                                                    | <b>3</b>  |
| <b>Глава 1. Технологический стек и архитектура проекта .....</b>         | <b>4</b>  |
| <b>1.1 Используемые технологии и инструменты.....</b>                    | <b>4</b>  |
| <b>1.2 Архитектура приложения .....</b>                                  | <b>4</b>  |
| <b>Глава 2. Модели данных.....</b>                                       | <b>5</b>  |
| <b>2.1 Обзор сущностей.....</b>                                          | <b>5</b>  |
| <b>2.2 Ключевые классы .....</b>                                         | <b>6</b>  |
| <b>Глава 3. Сервисный слой .....</b>                                     | <b>7</b>  |
| <b>3.1 Интерфейс IDataService&lt;T&gt;.....</b>                          | <b>7</b>  |
| <b>3.2 OrganizationService и WarehouseService .....</b>                  | <b>7</b>  |
| <b>3.3 ProductService и InvoiceService .....</b>                         | <b>7</b>  |
| <b>Глава 4. MVVM и слой ViewModel .....</b>                              | <b>8</b>  |
| <b>4.1 BaseViewModel.....</b>                                            | <b>8</b>  |
| <b>4.2 OrganizationViewModel.....</b>                                    | <b>8</b>  |
| <b>4.3 WarehouseViewModel и ProductViewModel.....</b>                    | <b>8</b>  |
| <b>4.4 InvoiceViewModel.....</b>                                         | <b>9</b>  |
| <b>Глава 5. Пользовательский интерфейс .....</b>                         | <b>9</b>  |
| <b>5.1 Главное окно MainWindow .....</b>                                 | <b>9</b>  |
| <b>5.2 Стилизация приложения (Styles.xaml) .....</b>                     | <b>10</b> |
| <b>5.3 ContextBarControl.....</b>                                        | <b>10</b> |
| <b>Глава 6. Модульное тестирование .....</b>                             | <b>11</b> |
| <b>6.1 Структура тестового проекта .....</b>                             | <b>11</b> |
| <b>6.2 Тесты OrganizationService .....</b>                               | <b>11</b> |
| <b>6.3 Тесты WarehouseService, ProductService и InvoiceService .....</b> | <b>12</b> |
| <b>Глава 7. Рабочий процесс приложения .....</b>                         | <b>12</b> |
| <b>Заключение .....</b>                                                  | <b>16</b> |

## **Введение**

Настоящая практическая работа посвящена проектированию и разработке настольного WPF-приложения, предназначенного для автоматизации складского учёта. Разработанная система позволяет вести учёт организаций, складов, товарной номенклатуры и накладных в рамках единого интерфейса. Приложение реализует полный цикл операций: создание и редактирование записей, проводку документов с автоматическим обновлением остатков, поиск и фильтрацию товаров, а также импорт данных из CSV-файлов.

Целью работы является разработка функционального десктопного приложения, демонстрирующего применение MVVM, слоистой организации кода и принципов объектно-ориентированного программирования на языке C# с использованием платформы WPF и .NET Framework 4.7.2.

В ходе разработки реализованы три взаимосвязанных функциональных модуля: управление организациями, управление складами, управление товарной номенклатурой. Дополнительно реализован модуль работы с накладными, обеспечивающий автоматическое изменение остатков при проводке документов. Для обеспечения качества кода разработан комплект модульных тестов с использованием фреймворка NUnit.

## Глава 1. Технологический стек и архитектура проекта

### 1.1 Используемые технологии и инструменты

Для реализации выбран стек на базе .NET Framework 4.7.2. Полный перечень технологий приведён в таблице 1.

Таблица 1 – перечень технологий

| Компонент                 | Технология / Инструмент                 |
|---------------------------|-----------------------------------------|
| Платформа разработки      | .NET Framework 4.7.2, язык C#           |
| UI-фреймворк              | WPF (Windows Presentation Foundation)   |
| Архитектурный паттерн     | MVVM (Model – View – ViewModel)         |
| Хранение данных           | Коллекции в памяти (ApplicationContext) |
| Модульное тестирование    | NUnit — тесты для каждого сервиса       |
| Среда разработки          | Visual Studio 2022                      |
| Язык разметки интерфейса  | XAML                                    |
| Дополнительные библиотеки | System.Runtime.Numerics (BigInteger)    |

### 1.2 Архитектура приложения

Приложение построено на трёхслойной архитектуре MVVM:

- слой моделей (WarehouseData.Models) — классы-сущности с аннотациями DataAnnotations;
- слой сервисов (WarehouseData.Services) — бизнес-логика и валидация;

— слой представления (Warehouse) — XAML-окна, ViewModel-классы, ресурсы стилей.

Хранилищем данных служит статический класс `ApplicationContext` с коллекциями `List<T>`, заполняемыми тестовыми данными при запуске через метод `Initialize()`. Это позволяет запускать приложение без базы данных и сосредоточиться на логике и архитектуре.

Точкой входа является класс `App`, переопределяющий событие `Startup`. Приложение сначала открывает окно выбора организации `OrganizationWindow`; если организация не выбрана — завершает работу; в противном случае открывает `MainWindow`.

## Глава 2. Модели данных

### 2.1 Обзор сущностей

В системе определено восемь классов-моделей, расположенных в пространстве имён `WarehouseData.Models` и снабжённых атрибутами `Table` и `Column` из `System.ComponentModel.DataAnnotations`. Перечень классов представлен в таблице 2.

Таблица 2 — Классы моделей данных

| Класс        | Назначение                            |
|--------------|---------------------------------------|
| Organization | Юридическое лицо — владелец складов   |
| Warehouse    | Склад, принадлежащий организации      |
| Product      | Товарная позиция на конкретном складе |

|              |                                           |
|--------------|-------------------------------------------|
| Invoice      | Документ поступления или списания товаров |
| InvoiceItem  | Строка накладной: товар, количество, цена |
| Category     | Справочник категорий товаров              |
| Manufacturer | Справочник производителей                 |
| Supplier     | Справочник поставщиков                    |

## 2.2 Ключевые классы

— Organization. Хранит OrgId (int, автоинкремент через ApplicationContext.NextOrgId()) и строку OrgName, помеченную [Required]. Содержит навигационную коллекцию ObservableCollection<Warehouse>.

— Warehouse. Хранит WhId, WhName, WhAddress и внешний ключ OrgId. Атрибут [ForeignKey(«OrgId»)] обеспечивает связь с Organization. Конструктор с тремя параметрами упрощает создание новых записей.

— Product. Центральная сущность системы. Поля Price и Discount имеют типы decimal и int соответственно, Quantity — int. Связи с Category, Manufacturer и Supplier реализованы как навигационные свойства без явного указания внешних ключей, что упрощает привязку данных в DataGridView.

— Invoice / InvoiceItem. Invoice хранит тип («Входящая» / «Исходящая») и список строк Items. Хранение Price непосредственно в InvoiceItem исключает искажение исторических данных при изменении текущей цены товара.

— Справочные классы (Category, Manufacturer, Supplier) содержат только Id и Name. Они загружаются при старте приложения в методе ApplicationContext.Initialize() и остаются неизменными в течение сессии.

## Глава 3. Сервисный слой

### 3.1 Интерфейс `IDataService<T>`

Все сервисы реализуют обобщённый интерфейс `IDataService<T>` с четырьмя методами: `GetAll()`, `Add(T item)`, `Update(T item)`, `Delete(T item)`. Единый интерфейс обеспечивает однородность API, упрощает модульное тестирование и соответствует принципу DRY (Don't Repeat Yourself).

### 3.2 `OrganizationService` и `WarehouseService`

`OrganizationService.Add()` выполняет валидацию поля `OrgName`: при пустом или состоящем из пробелов значении выбрасывается `ArgumentException`. Метод автоматически присваивает `OrgId` через `ApplicationContext.NextOrgId()`. Метод `Delete()` проверяет отсутствие привязанных складов — при их наличии выбрасывается `InvalidOperationException`, что исключает появление «осиротевших» записей.

`WarehouseService` дополнительно реализует метод `GetByOrganization(int orgId)`, используемый `WarehouseViewModel` для загрузки складов текущей организации. Метод `Delete()` аналогично проверяет отсутствие привязанных товаров перед удалением склада. Метод `Add()` проверяет аргумент на `null` и выбрасывает `ArgumentNullException` в соответствующем случае.

### 3.3 `ProductService` и `InvoiceService`

`ProductService.GetByWarehouse(int whId)` фильтрует товары по идентификатору склада. Метод `Add()` проверяет товар на `null`, пустое имя, отрицательную цену и отрицательный остаток. Метод `Update()` явно перечисляет все обновляемые поля: `Name`, `Price`, `Quantity`, `Discount`, `Category`, `Manufacturer`, `Supplier`, `PhotoPath`.

`InvoiceService` реализует ключевой метод `ProcessInvoice(Invoice invoice)`. При проводке входящей накладной (тип «Входящая») количество каждого

товара в списке Items увеличивается на указанное значение. При исходящей накладной («Исходящая») — уменьшается. Перед уменьшением проверяется достаточность остатка: при нехватке выбрасывается `InvalidOperationException` с именем проблемного товара, защищая от отрицательных остатков.

Метод `Update()` сервиса накладных сначала откатывает эффект предыдущей версии документа через приватный метод `ReverseInvoice()`, а затем применяет `ProcessInvoice()` к обновлённым данным. Это гарантирует корректность складских остатков при любом редактировании.

## **Глава 4. MVVM и слой ViewModel**

### **4.1 BaseViewModel**

Все `ViewModel`-классы наследуются от `BaseViewModel`, реализующего интерфейс `INotifyPropertyChanged`. Метод `OnPropertyChanged()` декорирован атрибутом `[CallerMemberName]`, что позволяет вызывать его без явного указания имени свойства и исключает ошибки при рефакторинге.

### **4.2 OrganizationViewModel**

`OrganizationViewModel` управляет `ObservableCollection<Organization>`. Метод `LoadData()` очищает коллекцию и перезаполняет её из сервиса без разрыва привязки `ItemsSource`. При редактировании организации используется пара `RemoveAt/Insert`, поскольку `ObservableCollection` не отслеживает изменения свойств элементов.

### **4.3 WarehouseViewModel и ProductViewModel**

`WarehouseViewModel` хранит поле `_currentOrgId` и вызывает `GetByOrganization()` при каждой смене организации. `AddWarehouse()`, `EditWarehouse()`, `DeleteWarehouse()` делегируют операции соответствующему сервису и синхронизируют коллекцию.



ProductViewModel при инициализации загружает справочники (Categories, Manufacturers, Suppliers) из ApplicationContext в ObservableCollection для заполнения ComboBox-ов диалога товара. Метод UpdateProduct() после обновления полностью перезагружает список товаров текущего склада.

#### **4.4 InvoiceViewModel**

InvoiceViewModel.AddInvoice() создаёт объект Invoice, заполняет список Items из ObservableCollection<InvoiceItem>, вызывает InvoiceService.ProcessInvoice() для изменения остатков и затем сохраняет накладную через InvoiceService.Add(). При возникновении InvalidOperationException вызывающий код (MainWindow) информирует пользователя через MessageBox.

EditInvoice() делегирует вызов InvoiceService.Update(), который обеспечивает атомарный откат старой проводки и применение новой. DeleteInvoice() вызывает InvoiceService.Delete(), который автоматически возвращает складские остатки в исходное состояние.

### **Глава 5. Пользовательский интерфейс**

#### **5.1 Главное окно MainWindow**

MainWindow (Height=620, Width=1100) является центральным окном приложения. Макет выполнен на трёхуровневом Grid: верхняя строка — меню, средняя — рабочая область, нижняя — строка контекста.

Рабочая область разделена GridSplitter на две панели: левая (280 px) содержит ListView складов, правая — DataGrid товаров с полосой поиска. Меню включает пять разделов: Файл, Склады, Товары, Накладные,

Справочники. Контекстные меню ListView и DataGrid дублируют основные операции.

Поиск товаров реализован в обработчике txtSearch\_TextChanged: при непустом тексте ItemsSource DataGrid заменяется отфильтрованным списком через LINQ Where(). При очистке строки поиска отображается полная коллекция ProductVM.Products.

## 5.2 Стилизация приложения (Styles.xaml)

Все стили вынесены в файл Resources/Styles.xaml и подключены в App.xaml через MergedDictionaries. Цветовая палитра включает основной синий (#0078D7), тёмно-синий (#005A9E) и красный цвет опасности (#C0392B). Шрифтом приложения выбран Segoe UI.

Определены три стиля кнопок: BaseButtonStyle (серый фон #E1E1E1), PrimaryButton (синий) и DangerButton (красный). Все кнопки имеют CornerRadius=2 и анимированные триггеры IsMouseOver / IsPressed. DataGrid использует чередование строк White / #F7F7F7 при высоте строки 44 px. Стили ListView, TabControl, GroupBox, Menu и других элементов управления обеспечивают единый визуальный стиль.

## 5.3 ContextBarControl

ContextBarControl — пользовательский элемент управления, отображающий в нижней части главного окна текущую организацию и склад. Фон панели — #F0F0F0, верхняя граница — #ABABAB. Название склада выделено синим цветом (#0078D7). Элемент содержит кнопку «Выбрать организацию» и публикует событие SelectOrganizationRequested.

Методы SetOrganization(string) и SetWarehouse(string) позволяют MainWindow обновлять строку контекста при каждой смене организации или склада. Null-значения обрабатываются с подстановкой текста по умолчанию.

## Глава 6. Модульное тестирование

### 6.1 Структура тестового проекта

Тестовый проект `WarehouseDataTests` содержит четыре тест-класса (`TestFixture`), охватывающих все сервисы слоя `WarehouseData.Services`. Каждый тест-класс снабжён методом `SetUp`, который перед каждым тестом очищает коллекции `ApplicationContext`, сбрасывает счётчики идентификаторов и создаёт необходимые тестовые данные (организацию, склад, товар). Это обеспечивает полную изоляцию тестов.

### 6.2 Тесты `OrganizationService`

Класс `OrganizationServiceTests` содержит тесты для всех методов сервиса. Перечень проверяемых сценариев:

- `GetAll` возвращает пустой список при пустом контексте;
- `GetAll` возвращает все добавленные организации;
- `Add` добавляет организацию в контекст и присваивает `Id > 0`;
- `Add` выбрасывает `ArgumentException` при пустом, пробельном и `null`-имени;
- `Add` гарантирует уникальность `Id` при добавлении нескольких записей;
- `Update` изменяет имя организации;
- `Update` выбрасывает `ArgumentException` при пустом и пробельном имени;
- `Update` не выбрасывает исключение для несуществующей записи;
- `Delete` удаляет организацию без складов;
- `Delete` выбрасывает `InvalidOperationException` при наличии привязанных складов и не удаляет запись.

### 6.3 Тесты WarehouseService, ProductService и InvoiceService

WarehouseServiceTests проверяет аналогичный набор сценариев для складов: добавление, редактирование, удаление с проверкой наличия товаров (InvalidOperationException), получение по организации.

ProductServiceTests содержит расширенный набор проверок метода Add(): null-товар (ArgumentNullException), пустое/пробельное имя (ArgumentException), отрицательная цена и отрицательный остаток (ArgumentException). Нулевые значения цены и остатка считаются допустимыми. Тесты Update() проверяют каждое изменяемое поле (Name, Price, Quantity, Discount) отдельно.

InvoiceServiceTests проверяет бизнес-логику проводки: входящая накладная увеличивает Quantity товара, исходящая — уменьшает. Тест на недостаточный остаток проверяет одновременно факт выброса InvalidOperationException и неизменность Quantity после исключения. Тесты Delete() проверяют откат эффекта проводки. Update() проверяется на корректное изменение номера накладной и безопасность при несуществующем Id.

## Глава 7. Рабочий процесс приложения

Приложение реализует строгую иерархию контекста: организация → склад → товары. Работа начинается с обязательного выбора организации ещё до открытия главного окна. Пользователь не может работать с товарами без предварительного выбора склада. Это отражает реальную структуру складского учёта и предотвращает создание «висячих» записей.

Этап 1: Запуск приложения. При старте вызывается ApplicationContext.Initialize(), который формирует тестовый набор данных: 10 категорий (Обувь, Одежда, Электроника, Мебель, Зоотовары, Еда,

Инструмент, Спорт товары, Аксессуары, Косметика), 10 производителей (Adidas, Nike, Bosch, Milka, Royal Canin, Диваны, Marmelato, Vivien Sabo, Indesit, Mixit), 2 поставщика, 2 организации, 3 склада и 5 товаров с различными атрибутами. Сразу после инициализации, до открытия главного окна, автоматически открывается OrganizationWindow.

Этап 2: Выбор организации (первый экран). OrganizationWindow открывается модально (ShowDialog) как первое и единственное видимое окно. Пользователь видит список всех организаций и может добавить, отредактировать или удалить любую из них через контекстное меню. Выбор организации выполняется двойным кликом или кнопкой ОК. Если пользователь закрывает окно без выбора (крестик или кнопка «Закрыть»), SelectedOrganization остаётся null, App\_Startup вызывает Shutdown() и приложение завершается — главное окно не открывается. При успешном выборе открывается MainWindow и вызывается mainWindow.SelectOrganization(), который загружает список складов выбранной организации.

Этап 3: Выбор склада в главном окне. После открытия MainWindow в левой панели ListView отображаются склады выбранной организации. При клике по складу (listWarehouses\_SelectionChanged) загружаются товары (ProductVM.LoadData) и накладные (InvoiceVM.LoadData) этого склада, строка контекста ContextBarControl обновляется — отображает название организации и склада. Пользователь может добавить новый склад (название и адрес вводятся через два последовательных InputDialog), отредактировать или удалить выбранный (удаление требует ввода точного названия склада для подтверждения). Смена организации доступна в любой момент через кнопку «Выбрать организацию» в строке контекста или через меню «Справочники → Организации» — при этом снова открывается OrganizationWindow.

Этап 4: Управление товарами. После выбора склада в DataGrid отображаются товары. Добавление и редактирование производится через ProductDialog с валидацией всех полей. Удаление требует подтверждения через MessageBox.Show(YesNo). Поиск работает мгновенно по TextChanged. Импорт из CSV доступен через меню «Склады → Импорт товаров».

Этап 5: Работа с накладными. Пункт меню «Накладные → Открыть список накладных» открывает InvoiceListWindow. В диалоге InvoiceDialog пользователь вводит номер накладной, выбирает тип (входящая/исходящая) и добавляет позиции (товар + количество). После нажатия «ОК» InvoiceViewModel.AddInvoice() проводит накладную: остатки пересчитываются немедленно. При нехватке товара пользователь видит MessageBox с описанием проблемы, накладная не сохраняется. После закрытия InvoiceListWindow MainWindow перезагружает список товаров для отражения изменённых остатков.

Таблица 3 — Доступность операций в зависимости от контекста

| Операция              | Условие доступности | Метод сервиса                  |
|-----------------------|---------------------|--------------------------------|
| Добавить организацию  | Всегда              | OrganizationService.Add()      |
| Добавить склад        | Организация выбрана | WarehouseService.Add()         |
| Добавить товар        | Склад выбран        | ProductService.Add()           |
| Импорт товаров из CSV | Склад выбран        | ProductService.Add() (в цикле) |

|                           |                             |                                 |
|---------------------------|-----------------------------|---------------------------------|
| Провести накладную        | Склад выбран                | InvoiceService.ProcessInvoice() |
| Удалить организацию       | Нет привязанных складов     | OrganizationService.Delete()    |
| Удалить склад             | Нет привязанных товаров     | WarehouseService.Delete()       |
| Удалить товар             | Товар выбран в DataGridView | ProductService.Delete()         |
| Редактировать организацию | Организация выбрана         | OrganizationService.Update()    |
| Редактировать склад       | Склад выбран в ListView     | WarehouseService.Update()       |
| Редактировать товар       | Товар выбран в DataGridView | ProductService.Update()         |
| Редактировать накладную   | Накладная выбрана в списке  | InvoiceService.Update()         |
| Удалить накладную         | Накладная выбрана в списке  | InvoiceService.Delete()         |

## Заключение

В ходе выполнения практической работы разработано настольное WPF-приложение «Warehouse» для автоматизации складского учёта, реализующее полный цикл управления организациями, складами, товарной номенклатурой и накладными.

В процессе разработки применён MVVM, обеспечивающий чёткое разделение ответственности между слоями. Сервисный слой реализован через обобщённый интерфейс `IDataService<T>`, что обеспечивает однородность API и упрощает тестирование. Бизнес-правила — валидация входных данных, защита от «осиротевших» записей, контроль складских остатков — сосредоточены в сервисах.

Разработан комплект модульных тестов на базе NUnit, охватывающих все четыре сервиса. Тесты выстроены по принципу полной изоляции через сброс `ApplicationContext` в методе `SetUp`. Покрыты как позитивные сценарии, так и граничные случаи (null-значения, пустые строки, недостаточный остаток, удаление связанных сущностей).

Пользовательский интерфейс выполнен на WPF с применением единой системы стилей (`Styles.xaml`). Реализованы поиск товаров в реальном времени, импорт из CSV, контекстные меню, строка контекста с отображением текущей организации и склада.

Таким образом, поставленные цели работы достигнуты: создано функциональное приложение, демонстрирующее практическое применение принципов ООП, MVVM и методологии модульного тестирования.